

Projet Python — Séance 2

Fonctions, gestion des erreurs, journalisation et récursivité

Enseignante : Rahma GARGOURI — 2026-2027

Séance de 3 heures — Projet en groupe

Objectif. Organiser son code en **fonctions**, apprendre à **gérer les erreurs** (`try/except`) et à **journaliser** (`logging`) à l'aide de la bibliothèque standard, puis découvrir la **récursivité** et la notion de **complexité** (temps et mémoire).

Comme en séance 1, le travail se fait sur une branche `seance-2`, avec une *pull request* en fin de séance. Après chaque petite partie, une boîte bleue rassemble des questions à **rédigier** dans `docs/notes_seance2.md` et à **commiter**. L'IA est autorisée, mais chaque réponse doit être comprise (elle pourra être demandée à l'oral).

Partie A — Les fonctions

A.1 — Écrire vos premières fonctions. Reprendre les données de la séance 1 (le fichier CSV des taux) et découper le traitement en **fonctions**, par exemple :

- une fonction qui lit le CSV et renvoie une liste de couples (`date`, `taux`) ;
- une fonction qui calcule la **moyenne** d'une liste de taux ;
- une fonction qui renvoie le taux **minimum** et le taux **maximum**.

Chaque fonction a des **paramètres**, une valeur de retour (`return`) et une **docstring** (courte description).

A.2 — Utiliser une fonction d'une bibliothèque. Recalculer la moyenne avec la fonction `mean` du module `statistics` (bibliothèque standard) et comparer avec votre propre fonction.

Questions — à rédiger et à commiter

1. Pourquoi découper le code en fonctions ? Citez trois bénéfices concrets.
2. Quelle différence entre un **paramètre** et la **valeur de retour** (`return`) ? Que renvoie une fonction sans `return` ?
3. Qu'est-ce qu'une **docstring** et où la place-t-on ?
4. Quelle différence entre une variable **locale** (définie dans une fonction) et une variable **globale** ?

Partie B — Gérer les erreurs (`try/except`) et journaliser (`logging`)

B.1 — `try/except`. Reprendre l'appel à l'API de la séance 1 et l'entourer d'un `try/except` pour gérer les erreurs réseau : pas de connexion, code HTTP d'erreur (comme le 403 rencontré), réponse illisible. Afficher un **message clair** au lieu de laisser le programme s'arrêter brutalement.

B.2 — `logging`. Remplacer les `print` de suivi par du `logging` : configurer un fichier de log, puis utiliser les **niveaux** (`info` pour le déroulement normal, `warning` pour un cas suspect, `error` pour une erreur).

Questions — à rédiger et à commiter

1. Pourquoi gérer les erreurs avec `try/except`? Que se passe-t-il si une exception n'est pas gérée? À quoi servent `else` et `finally`?
2. Pourquoi un `except:` nu (sans préciser le type d'erreur) est-il déconseillé?
3. C'est quoi `logging`? Pourquoi le préférer à `print` dans un vrai programme? Citez les niveaux de log et un usage de chacun.

Partie C — Récursivité et complexité

C.1 — Une fonction récursive. Écrire une fonction récursive simple, par exemple la **factorielle** $n!$ ou la **somme** des éléments d'une liste. Identifier le **cas de base** (qui arrête la récursion) et le **cas récursif** (la fonction s'appelle elle-même).

C.2 — Quand la récursivité coûte cher. Écrire la suite de **Fibonacci** de façon récursive naïve. Mesurer le temps d'exécution (`time.perf_counter`) pour des valeurs croissantes : il explose. Améliorer avec la **mémoïsation** (`functools.lru_cache`), puis constater qu'une version **itérative** est tout aussi efficace. Pour une profondeur trop grande, observer la `RecursionError`.

C.3 — La complexité. Introduire la notion de complexité :

- **temporelle** : le nombre d'opérations en fonction de la taille n de l'entrée — par exemple $O(n)$ pour une boucle simple, $O(n^2)$ pour deux boucles imbriquées, $O(2^n)$ pour Fibonacci naïf;
- **spatiale** : la mémoire supplémentaire utilisée — par exemple la profondeur des appels récursifs, ou la mémoire occupée par la mémoïsation.

Comparer, **chiffres à l'appui**, les trois versions de Fibonacci (temps et mémoire).

Questions — à rédiger et à commiter

1. C'est quoi la **récursivité**? Cas de base et cas récursif? Que se passe-t-il si le cas de base est absent?
2. Pourquoi utiliser la récursivité? Est-elle **toujours** le meilleur choix? Donnez un **contre-exemple** précis.
3. C'est quoi la complexité **temporelle**? La complexité **spatiale**? Que signifient $O(n)$, $O(n^2)$, $O(2^n)$?
4. C'est quoi la **mémoïsation**? Que fait `functools.lru_cache`? Pourquoi Python limite-t-il la profondeur de récursion?

Contraintes imposées

Cette séance reste en **bibliothèque standard** (`statistics`, `logging`, `functools`, `time`, `urllib`, `json`...). Les librairies `pandas` et `numpy` seront découvertes à la séance 3.

Livrables de la séance (commit + push + PR avant la fin)

`fonctions.py` (lecture du CSV, moyenne, min/max) — l'appel à l'API protégé par `try/except` et journalisé (`logging`, fichier `.log`) — `recursivite.py` (factorielle/somme + les trois versions de Fibonacci avec mesures de temps) — `docs/notes_seance2.md` (réponses aux questions + tableau des mesures) — **PR de la séance 2** ouverte.